

Quorum — multi-issuer RWA vaults on Solana

Technical specification, v1 (STOCKLANA hackathon build) Product name: **Quorum**, tokens **qNVDA** , **qSPY** , **qGOLD** Draft: 2026-09-19 · Deadline: 2026-09-25, 4:00 PM ET

0. Name and positioning

Quorum. A quorum is the minimum number of members needed for a decision to count. No single issuer decides what your NVDA is worth here, the basket does. Tokens are **qNVDA** , **qSPY** , **qGOLD** .

Category line: issuer-diversified RWA vaults. Not an index (we hold one asset), not an ETF (we don't hold the raw asset, we hold other people's wrappers of it).

The TradFi analogy to use with non-crypto people: an insured cash sweep. Banks spread a deposit across many institutions so no single failure hurts you, mechanically and in the background. Quorum does that for tokenized stocks. Other reference points: a fund holding SPY, IVV and VOO together, ADR-versus-ordinary arbitrage, and multi-custodian bullion funds for **qGOLD** .

Names rejected: "Alloy" and an **a** prefix, because Tether already ships Alloy with **aUSDT** , and **aTokens** read as Aave deposits to every DeFi user.

1. Summary

Every real-world asset on Solana is issued by several competing wrappers (xStocks, Ondo, Backpack, Paxos, Tether and others). Each wrapper has its own liquidity, its own premium or discount, its own dividend mechanics and its own issuer risk. Users have to pick one, and they take all of that wrapper's risk.

The Quorum holds **every major wrapper of one asset** and issues a single token against the whole basket. One **qNVDA** is a claim on a diversified bundle of NVDA wrappers, priced by oracle NAV instead of any single DEX pool.

Why it beats a brokerage account (the STOCKLANA brief):

- 24/7 mint and redeem, with guard rails when the underlying market is closed
- No single-issuer risk: caps per issuer, automatic quarantine on depeg
- Best execution across all wrappers, and the arbitrage spread goes to holders
- One token that composes into any Solana DeFi protocol

2. v1 scope

Three vaults, three wrappers each, one deposit asset.

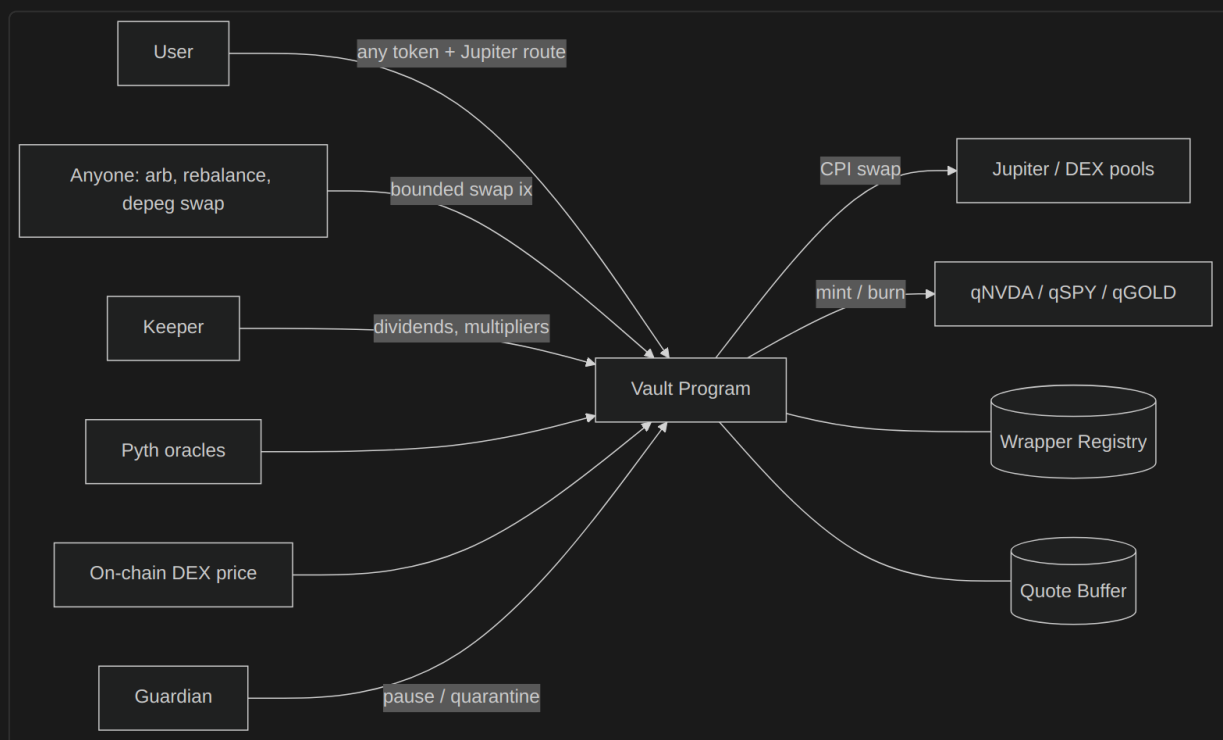
Vault	Wrapper 1	Wrapper 2	Wrapper 3	Unit of account
qNVDA	xStocks NVDAx	Ondo NVDAon	Backpack Securities NVDA	1 NVDA share
qSPY	xStocks SPYx	Ondo SPYon	Backpack Securities SPY	1 SPY share
qGOLD	Paxos PAXG	Tether XAUt0	Oro GOLD	1 troy ounce

- **Deposit asset: anything.** The user can pay with any token. The frontend fetches a Jupiter quote and route, and the swap runs inside the same transaction as the mint. The program only ever credits a whitelisted wrapper, so the deposit token itself never has to be trusted.
- **Mint modes:** in-kind (deposit a whitelisted wrapper directly) and swap-and-mint (any token, atomic, quote taken on the site).
- **No request queue.** Every user action settles in one transaction.
- **Out of scope for v1:** USDT/SOL deposits, more than 3 wrappers per asset, governance, pre-IPO baskets, and cross-asset baskets.

Gold note: the three gold wrappers are all 1 token = 1 troy ounce, so the unit is clean. Gold ETF wrappers (GLDx , GLDon) are left out of v1 on purpose, because one GLD share is a slowly decaying fraction of an ounce. That's a v2 item with its own conversion oracle.

Verify before coding: each mint address, Token-2022 extensions, transfer restrictions, and whether a program-owned account (PDA) is allowed to hold the token. See section 11.

3. Architecture



Components

1. **Vault program (Anchor):** holds all wrapper balances in PDA token accounts, keeps the registry and NAV state, mints and burns the index token, CPIs into Jupiter for swaps, and enforces every guard on-chain.
2. **Keeper (privileged, minimal):** sweeps cash dividends and pushes corporate-action multipliers. It is never in the path of a user mint, so it cannot censor or delay anyone.
3. **Arb, rebalance and depeg swaps are permissionless.** Anyone can call them once the on-chain conditions in sections 7 and 9.1 hold. Every swap is bounded by the program, so a hostile caller cannot drain value.
4. **Guardian:** a multisig or monitor bot that can pause and quarantine. It can only make things more restrictive, never move funds.
5. **Dashboard:** NAV, the per-wrapper breakdown, live premium or discount per issuer, guard status, and mint/redeem.

4. On-chain accounts

Vault (one per asset) - `index_mint`, `quote_buffer` (USDC token account), `authority`, `guardian` - `unit` (SHARE or OUNCE), `underlying_oracle` (Pyth feed of the real asset) - `buffer_target_bps` (default 300 = 3%, only used to smooth small redeems), `fee_mint_bps`, `fee_redeem_bps` - `swap_whitelist`: the set of mints this vault may ever swap into or out of,

which is its own wrappers plus the approved quote assets (USDC, SOL). See 6.4. - `status` (ACTIVE, MARKET_CLOSED, PAUSED) - `nav_per_token` , `nav_updated_slot` `WrapperConfig` (one per wrapper per vault) - `wrapper_mint` , `vault_token_account` (PDA-owned) - `units_per_token` : fixed point, the underlying shares or ounces that one wrapper token represents (it changes with corporate actions, see section 8) - `multiplier_source` : FIXED, TOKEN2022_SCALED_UI, or KEEPER_PUSHED - `target_weight_bps` , `max_weight_bps` (the issuer cap, default 4000 = 40%) - `dex_price_feed` : DEX TWAP source used for depeg checks - `status` : ACTIVE, MINT_DISABLED, QUARANTINED - `haircut_bps` : applied to NAV while QUARANTINED

There are no request accounts. Mint and redeem are single-transaction instructions, so there is nothing queued and nothing to settle later.

5. NAV math

For each vault:

```
unit_price      = Pyth price of the real asset (NVDA, SPY, XAU)
wrapper_units_i = balance_i * units_per_token_i * (1 - haircut_i)
total_units     = sum(wrapper_units_i)
NAV_total       = total_units * unit_price + quote_buffer
nav_per_token   = NAV_total / index_supply
```

- NAV **never** uses DEX prices. DEX prices are used only to detect depegs and as a sanity band.
- Pyth confidence interval: if `conf / price > max_conf_bps` , mint and redeem in cash mode are blocked until it tightens.
- Staleness: reject any price older than `max_age` (for example 60s while the market is open).

6. Flows

6.1 In-kind mint (build first)

1. The user deposits `x` of whitelisted wrapper `i` .
2. Checks: wrapper `i` is ACTIVE, the post-deposit weight stays within `max_weight_bps` , and the oracle is fresh.
3. `units = x * units_per_token_i` , and `value = units * unit_price` .
4. Mint `value / nav_per_token * (1 - fee_mint)` index tokens.

This path needs no swap and has zero slippage. It's also how arbitrageurs push wrapper prices back to the peg.

6.2 Swap-and-mint (any deposit token)

1. The frontend quotes the route with the Jupiter API: `deposit token -> target wrapper`, where the target is the most under-weight ACTIVE wrapper. It shows the user the expected index tokens and the price impact.
2. The user signs one transaction carrying that route plus `min_units_out`.
3. The program: - checks the **input mint** is allowed (any token is fine here, but it must not be one of this vault's own wrappers unless the call is an in-kind mint) - checks the **output mint** is a whitelisted ACTIVE wrapper of this vault (section 6.4) - CPIs into Jupiter with the supplied route - measures the **real balance delta** of the vault's wrapper account before and after, and uses that, never the quoted number - rejects if `units_received < min_units_out` - prices the received units at the oracle and mints `value / nav_per_token * (1 - fee_mint)` to the user
4. Everything is atomic. A bad route, a moved price or a hostile Jupiter result fails the whole transaction, and the user keeps their deposit.

Because pricing uses the measured delta valued at the oracle, a user who brings a bad route only hurts themselves: they receive fewer index tokens. Existing holders are never diluted.

6.3 Redeem

- **In-kind:** burn and receive pro-rata wrappers. Always available, including when the market is closed. This is the "exit is always open" guarantee.
- **Swap-and-redeem:** burn, and the program sells wrappers (over-weight first) through a user-supplied route into the token the user wants, with `min_amount_out` enforced. Same whitelist checks, same measured-delta accounting.
- Small redeems can be paid straight from the quote buffer with no swap.

6.4 Swap whitelist (both legs)

Every swap the program performs, whoever calls it, is checked on both sides:

```
allowed_out(vault) = { wrappers of this vault with status ACTIVE }
allowed_quote      = { USDC, SOL }                               // approved quote assets

mint path: input = any mint,                                     output in allowed_out
redeem path: input in vault wrappers,                             output in allowed_out + allowed_quote + user's ch
rebalance:  input in vault wrappers,                             output in allowed_out
depeg swap: input = the depegged wrapper,                         output in allowed_out + allowed_quote
```

So the vault can sell `NVDax` for `NVDAon` or for USDC, but it can never end up holding SHIB or `AAPLx`. The rule is enforced against the **registry**, not against anything the caller passes in, and quarantined wrappers are removed from `allowed_out` automatically so nobody can route value into a broken issuer.

6.5 Backpack issuer route (optional, v1.5)

Backpack Securities opened a mint and redeem API to Solana developers, with tokens 1:1 redeemable for the real share. For large flows an operator can mint straight from the issuer instead of buying on a DEX. It needs a KYC'd operator account, so it stays out of v1.

7. Rebalance and arbitrage bot

Target weights: proportional to each wrapper's 7-day DEX liquidity, capped at `max_weight_bps` per issuer, with a 15% floor so every issuer stays represented.

Rebalance trigger (checked on-chain, anyone may call): a wrapper drifts more than 5 percentage points from target, or a wrapper changes status. The caller supplies the route, the program bounds the result, and a small share of the improvement can be paid to the caller as a keeper reward.

Arbitrage loop (the core value engine):

```
for each pair (i, j):
    prem_i = dex_price_i / (units_per_token_i * unit_price) - 1
    if prem_i - prem_j > threshold + fees:
        sell wrapper i, buy wrapper j    (inside weight caps)
```

- The spread earned goes into NAV, so holders capture it.
- **On-chain `rebalance` instruction:** the caller passes a swap route, and the program checks `total_units_after >= total_units_before * (1 - max_loss_bps)` using measured balance deltas. Nobody can make the basket lose units beyond that bound, so the instruction is safe to leave permissionless.
- Rebalance is disabled while the market is closed (the oracle is flat and premiums aren't meaningful).

8. Corporate actions (dividends, splits, rebase)

Verified 2026-09-19 from each issuer's own docs. The headline: **none of these wrappers pays a cash dividend that drops the token price.** Both stock issuers reinvest, so every wrapper stays a total-return tracker of the same share, and they should all trade at roughly the same price. What differs between them is the **unit count** behind one token, which is exactly what `units_per_token` captures.

Issuer	Mechanism	What the vault must do
xStocks (Backed)	Token-2022 Scaled UI Amount . Every xStock starts at multiplier 1.0. A dividend is reinvested net of 30% US withholding and the multiplier rises by <code>net_dividend / prior_close</code> . Splits move the same multiplier.	Read the multiplier from the mint extension, or equivalently read the scaled UI amount rather than the raw balance. <code>multiplier_source = TOKEN2022_SCALED_UI</code> .
Ondo Global Markets	Total-return tracker. Dividends are reinvested net of withholding and appear as more tokens in the holder's account. The price stays quoted in USD.	Nothing to push. The vault's balance simply grows, and <code>units_per_token</code> stays 1. Reconcile after each ex-date.
Backpack Securities	Not published. Tokens are 1:1 redeemable for the real share, so the payout is probably cash or a broker-side reinvest. Verify on day 1 .	If cash: a keeper sweeps the payout into the buffer, which lifts NAV. If reinvested as tokens: same as Ondo.
PAXG, XAUt0	No dividend. 1 token = 1 troy ounce, and storage fees are zero for PAXG.	<code>units_per_token</code> fixed at 1, <code>multiplier_source = FIXED</code> .
Oro GOLD	Marketed as yield-bearing gold, so confirm whether the yield is a rebase or a separate reward token.	Treat as <code>KEEPER_PUSHED</code> until confirmed, or leave it out of v1 if the mechanism is opaque.

Implementation trap: for xStocks, reading the **raw** SPL balance instead of the scaled amount makes the vault believe it holds fewer shares than it really does. NAV would then drift lower with every dividend, and mints would issue too many index tokens. This is the single most likely source of a silent accounting bug in the whole design, so it gets a dedicated test.

Why prices can still differ between wrappers

1. **Premium or discount from liquidity.** Thin pools drift further from fair value, and off-hours drift is larger. This is what the arb engine harvests.
2. **Withholding and fee drag.** Different issuer structures reinvest slightly different net amounts, which compounds into a small tracking difference over time. It shows up as a different unit count per token, not as a price gap.
3. **Perceived issuer risk.** A wrapper that people trust less trades cheaper.

The vault prices everything as `units_per_token x oracle price`, never as "1 token = 1 share", so all three drift independently and NAV stays correct.

Guards: - A `units_per_token` change larger than 1% needs guardian co-sign (this catches bad keeper pushes and unannounced splits). - Mint and redeem pause around a known split date until the new multiplier is confirmed on-chain. - A daily reconciliation job compares each wrapper's implied units against the issuer's published multiplier or supply, and alerts on any drift.

9. Guards and depeg handling

9.1 On-chain depeg test (permissionless swaps)

A depeg has to be **provable on-chain**, or the swap can't safely be left open to anyone. The test uses only accounts the program reads itself:

```
implied_i = units_per_token_i * unit_price    // oracle fair value of 1 wrapper token
dex_i     = TWAP from the registered pool of wrapper i
dev_i     = dex_i / implied_i - 1             // signed deviation
```

Sources for `dex_i`, in order of preference:

1. **A Pyth feed for the wrapper itself**, where one exists. Cleanest, and it comes with a confidence interval.
2. **A registered pool TWAP** (Metora, Orca or Raydium), read straight from the pool account. Register one canonical pool per wrapper at listing time and use a time-weighted price, never the spot tick, so one sandwiched block can't fake a depeg.

Conditions for a permissionless `swap_depegged` call:

Check	Rule
Deviation	<code>dev_i</code> at or past <code>soft_threshold</code> in either direction, default 2%
Persistence	It has held for <code>min_duration</code> (default 10 min) in the TWAP window
Oracle health	<code>unit_price</code> is fresh and its confidence is inside <code>max_conf_bps</code>
Direction	Sell the rich wrapper, buy the cheap one, never the reverse
Whitelist	Both legs pass section 6.4
Bound	<code>total_units_after >= total_units_before * (1 + min_gain_bps)</code> , measured, so the swap must leave the vault holding more underlying units
Size	Capped per call and per epoch (<code>max_swap_bps</code> , default 10% of the vault), with a cooldown
Caller reward	A small share of the realised gain, for example 10%, paid in index tokens

Because the instruction only runs when the deviation is real, can only trade whitelisted wrappers, and must end with more units than it started with, opening it to the public is strictly better than a private keeper. It removes an operator dependency and the vault gets defended the moment a wrapper slips.

Hard depeg (`dev_i` past `hard_threshold`, default 5%, or the issuer freezes the token) needs no caller: any instruction that touches the vault flips that wrapper to QUARANTINED, applies the NAV haircut, and drops it from `allowed_out`.

9.2 Guard table

Guard	Trigger	Action
Depeg soft	<code>dev_i</code> past 2% for 10 min (see 9.1)	Wrapper goes MINT_DISABLED, and <code>swap_depegged</code> opens to anyone
Depeg hard	More than 5%, or the issuer freezes or pauses the token	QUARANTINED: haircut applied in NAV, excluded from redeems, bot sells it down when safe
Issuer cap	Weight above <code>max_weight_bps</code>	Block in-kind deposits of that wrapper
Oracle stale / wide	Age over <code>max_age</code> , or conf over <code>max_conf_bps</code>	Block swap-and-mint, rebalance and depeg swaps. In-kind redeem stays open
Market closed	Pyth market-hours flag (weekends, holidays)	Status MARKET_CLOSED: higher fees, swap-and-mint size capped, no rebalance. Gold trades 24/5 so it gets its own calendar
NAV circuit breaker	NAV moves over 8% within 10 min	Global pause, guardian review
Permanent delegate / freeze	The wrapper has these Token-2022 extensions	Flagged at registration; monitor bot watches and quarantines on use

Rule: a guardian can pause and quarantine but can never withdraw funds. Unpausing needs the full authority multisig.

10. Fees (v1 defaults)

- Mint 0.10%, redeem 0.10%
- Market-closed surcharge +0.30% on swap-and-mint and swap-and-redeem
- Depeg-swap caller reward: 10% of the realised gain
- No management fee in v1
- Rebalance and arb profits accrue to NAV

11. Risks and open items

Must verify per wrapper (day 1): 1. Mint addresses and token program (SPL or Token-2022) with its extensions 2. Transfer restrictions or allowlists, and whether a PDA can hold the token 3. Geo and KYC terms (most wrappers exclude US persons). The index token inherits the strictest restriction. 4. The exact dividend mechanism for Ondo, Backpack and Oro 5. Pyth feed IDs for NVDA, SPY and XAU/USD, plus market-hours behavior 6. Whether Pyth publishes a feed for any

wrapper itself. If not, pick the canonical pool per wrapper for the TWAP in 9.1 7. Jupiter CPI limits: transaction size, compute budget, and how many route hops fit alongside the mint instruction

Structural risks: - **Regulatory:** a token backed by a basket of securities looks like a fund. That's fine for a hackathon, but it needs legal review before any real launch. - **Issuer seizure:** permanent-delegate wrappers can remove tokens from the vault. Issuer caps limit the damage. - **Liquidity:** thin pools on some wrappers make cash mint expensive, which is why there's a slippage bound and in-kind first. - **Weekend gap:** NAV is flat while the market is closed, but wrapper DEX prices move. That's covered by the MARKET_CLOSED status and in-kind-only redeem at size.

11b. Deployment and hosting

Piece	Where	Notes
Vault program	Solana mainnet	Anchor, upgrade authority on the team multisig for the hackathon
Frontend	Vercel	Next.js. API routes proxy Jupiter, Pyth Hermes and the RPC so no key reaches the browser
Keeper (dividends, multipliers)	VPS	systemd timer or cron
Depeg watcher	VPS	Runs every 30 to 60 seconds
Arb and rebalance bot	VPS	Long-running loop under pm2 or systemd
Guardian monitor	VPS	Alerts to Telegram, can call pause

Decision (2026-09-20): every bot job runs on our own VPS, none on Vercel. Vercel keeps only the user-facing app. The reasons: serverless functions cap out at 60s (Hobby) or 300s (Pro) so a tight arb loop does not fit, cron granularity is coarse, and keeping one wallet and one set of RPC credentials in one place is simpler to secure. A single `bots/` workspace with shared config also means one deploy, one log stream, and one place to rotate keys.

Bot workspace layout

```
bots/
  src/keeper.ts      // dividends, multiplier pushes
  src/watcher.ts     // depeg detection, guard alerts
  src/arb.ts         // permissionless rebalance + swap_depegged caller
  src/guardian.ts    // pause on circuit breaker, Telegram alerts
  src/shared/        // program client, oracle reads, config
  ecosystem.config.js // pm2
```

Operational rules: each bot uses its **own hot wallet** with a small balance (never the authority or guardian key), logs every submitted transaction signature, exits non-zero on repeated failure so pm2 restarts it, and posts a heartbeat to Telegram so a silent death is visible.

Note that the arb bot is a convenience, not a dependency. `swap_depegged` and `rebalance` are permissionless, so if our VPS dies, anybody else can still defend the vault, and the protocol keeps working.

12. Hackathon tracks

- **Main track (\$100k):** it's directly the brief, "better than a brokerage".
- **Pyth (Pro access):** NAV, confidence and market-hours all run on Pyth.
- **Clawpump (\$5k, agent-native trading):** frame the arb and rebalance bot as an autonomous agent (needs Meteora infrastructure, so check fit).

13. Build plan (6 days)

Day	Deliverable
1 (Sep 20)	Verify wrappers, pools and feeds (section 11). Anchor workspace, registry + vault accounts
2 (Sep 21)	In-kind mint and redeem, NAV with Pyth and <code>units_per_token</code>
3 (Sep 22)	Swap-and-mint and swap-and-redeem: Jupiter CPI, whitelist checks, measured deltas, <code>min_out</code>
4 (Sep 23)	Guards, the on-chain depeg test (9.1), permissionless <code>swap_depegged</code> and <code>rebalance</code> with bounds
5 (Sep 24)	Frontend (quote, NAV breakdown, live premium per issuer, guard status). Deploy to mainnet and seed each vault with small real positions
6 (Sep 25)	Real mainnet demo run, video, README, submission before 4:00 PM ET

Demo script (mainnet, real tokens, small size): mint `qNVDA` paying with SOL in one transaction with the quote taken on the site, then show NAV split across three issuers, then show a live premium or discount between two wrappers and have a **third-party wallet** call `swap_depegged` and collect the reward, then redeem in-kind while the market is closed.

Mainnet safety for the demo: per-vault position caps, a low per-transaction size limit, and the guardian multisig on the team's keys. Small real positions beat a devnet mock, because judges can verify every account on-chain.

14. Open questions for the team

1. The product name and token tickers

2. The operator for the Backpack issuer route (KYC account) in v1.5
3. Who is the guardian (a multisig of team members, or a bot with limited rights)?
4. The caller reward percentage for depeg swaps, and whether rebalance pays one too
5. How much real capital seeds each vault on mainnet for the demo